



Output Control

Output Control

1) What it is & why it matters

Output control is the set of techniques that shape *how* an LLM answers—its **length**, **format**, **style**, and **content boundaries**. In production, tight control improves:

- **Reliability** (consistent shape for downstream systems)
 - **UX quality** (tone, reading level, concision)
 - **Safety/compliance** (stay within allowed topics)
 - **Cost/latency** (no verbose tangents)
-

2) The Output-Control Stack (from strongest to weakest)

1. Constrained decoding (if available)

JSON/grammar modes, tool/function calling, logit bias/banlists.

2. Schema/contract

JSON Schema, TypeScript types, EBNF grammar; validated post-run.

3. System & task instructions

Role, audience, tone, format rules, style guides.

4. Decoding parameters

`max_tokens`, `stop`, `temperature`, `top_p`, `top_k`, penalties.

5. Post-processing

Validators, regex, linters, normalizers, safety filters.

Rule: Push as much control up the stack as your provider supports (schema / function calling), then reinforce with prompt + parameters, and finally catch issues with validators.

3) Parameter-level controls (model knobs)

3.1 Length & boundaries

- `max_tokens` → hard cap on length (prevents runaways; also caps cost/latency).
- `stop sequences` → precise end boundaries (e.g., `"\n\nEND"`, or `"]]"`).
 - *Tip*: Use a **unique sentinel** unlikely to occur naturally.
 - *Pitfall*: Over-aggressive stop strings can cut off mid-token or mid-JSON.

3.2 Style & variability

- `temperature` → randomness; lower for precision, higher for creativity.
- `top_p` / `top_k` → scope of token selection; lower for focus, higher for diversity.
 - *Tip*: Tune one primary knob (often `top_p`) and keep the other neutral.

3.3 Repetition & topical drift

- **Presence / frequency penalties** (if supported) → reduce repetition and nudge novelty.
 - *Heuristic*: presence penalty ~0.2–0.6 to avoid loops in long generations.

3.4 Token-level steering

- **Logit bias / token ban** (if supported) → boost or block exact tokens (e.g., forbid `"`"` or certain labels).
 - *Use sparingly*—can push model into awkward paraphrases.

3.5 Reproducibility

- `seed` (if supported) → more repeatable outputs for testing (not perfectly deterministic across versions).
-

4) Prompt-level controls (instruction design)

4.1 Role, audience, tone

Specify persona, reading level, voice.

Example: "You are a healthcare writer. Audience: adult patients. Tone: calm, plain language."

4.2 Format contracts

State *exact* structure: JSON keys, Markdown headings, bullet style, table columns.

4.3 Delimiters & sentinels

Fence user data with clear markers to avoid bleed:

"Use only facts inside `<<DATA>>...</DATA>`."

4.4 Few-shot structure exemplars

Show 1–3 **good outputs** with the exact shape you want.

4.5 Negative directives

Disallow content/classes: "Do not include code blocks," "No external links," "No emojis."

5) Strong-format strategies (production-ready)

5.1 JSON-only responses (with validation)

Prompt snippet

```
System: You are a structured-output generator.  
User: Return ONLY valid JSON that conforms to this schema.  
Do not include code fences or commentary.
```

```
Schema (TypeScript type):  
type ProductCard = {  
  id: string; title: string; price: number;  
  tags: string[]; shortDescription: string;  
};
```

Task: Create 3 ProductCard objects for entry-level laptops.
NO extra fields.

Post-processing

- Parse JSON → validate against **JSON Schema** → reject/repair if invalid.
- If your provider supports **JSON mode / function calling**, use it instead of free-form text.

5.2 Markdown sections

Prompt snippet

Write exactly these sections:

Overview

Pros

Cons

Action Items

- Keep each section ≤ 80 words.
- No additional sections or preambles.

5.3 Table shape (Markdown)

Produce a Markdown table with exactly these columns:

| Metric | Target | Result | Status |

- 5 rows max
- No narrative outside the table

5.4 Multi-part boundaries with **stop**

Generate two parts separated by the line:

=== END PART 1 ===

Stop generating immediately after:

=== END PART 2 ===

API

- Set `stop: ["=== END PART 2 ==="]`.
 - *Note:* Still validate; models may try to add trailing whitespace.
-

6) Safety & content boundaries

- **Topical constraints:** "Do not provide medical diagnosis. If asked, reply with: 'I'm not a medical professional...' and suggest seeing a doctor."
 - **Redaction:** "If PII appears, replace with `[REDACTED]`."
 - **Deflection templates:** Provide **verbatim** fallback messages for disallowed requests.
 - **Guardrail checkers:** Run outputs through policy classifiers before display.
-

7) Testing & evaluation (CI for prompts)

- **Golden tests:** Fixed inputs → assert shape (schema), sections, max length.
 - **Fuzz prompts:** Randomly vary wording/ordering to test robustness.
 - **Metrics to track:**
 - *Schema validity rate (%)*
 - *Constraint adherence* (length, sections present)
 - *Toxicity/safety flags* (count)
 - *Latency & token usage*
 - **A/B decoding:** Compare (`temperature=0.2, top_p=0.9`) vs (`0.0, 0.6`) on the same prompt set.
-

8) Troubleshooting guide

Symptom	Likely Cause	Fix
Extra prose around JSON	Model defaults to chatty preambles	Use JSON mode / function calling; say "NO preamble"; enforce regex strip before parsing
Invalid or partial JSON	Stop too early / high randomness	Lower <code>temperature</code> ; raise <code>max_tokens</code> ; supply minimal working example

Symptom	Likely Cause	Fix
Ignored length limits	Goal conflicts with word cap	Reorder instructions; put length limit earlier; use summarization step
Repetition loops	High temperature w/o penalties	Add presence/frequency penalty; add "avoid repeating phrases"
Drifts off-topic	High <code>top_p</code> / <code>top_k</code> ; vague task	Reduce <code>top_p</code> ; add tighter task statement & examples

9) Quick recipes

Structured report (safe & short)

- `temperature=0.2`, `top_p=0.6`, `max_tokens` conservative
- JSON mode or strict Markdown sections
- Post-validate with schema

Creative brainstorm (diverse)

- `temperature=0.9`, `top_p=0.95`, no `top_k` (or high)
- No rigid schema; cap with `max_tokens`
- Generate 5–10 samples and curate

Step-by-step reasoning (accurate)

- `temperature=0.0–0.2`, `top_p=0.5–0.7`
- Prompt: "Think step-by-step. Show numbered reasoning then final answer."
- Optionally strip the reasoning before display

10) Copy-paste checklist

- ☐ **State the format contract** (JSON schema or headings)
- ☐ **Pin length** (`max_tokens`) and **boundaries** (`stop`)
- ☐ **Tune randomness** (`temperature`, `top_p`) for task type
- ☐ **Forbid extras** (no preamble, no links, no emojis—if needed)
- ☐ **Provide an example** with the exact shape you expect

- ☐ **Validate outputs** (schema/regex/safety checks)
 - ☐ **Log & test** (goldens, fuzzing, adherence metrics)
-

Key takeaway

Effective output control is **contract + parameters + validation**. Specify the shape you need, reinforce it with decoding settings, and verify it automatically—this is what turns LLMs from clever chat partners into **reliable system components**.